Автоматическая генерация признаков для рекомендаций

Учебный исследовательский проект · Yambda-50M

Python · pandas · NumPy · LightGBM · Optuna · scikit-learn

В этом проекте я исследую, может ли генерация агрегатов по шаблону заменить ручной выбор временных окон. Я реализовал генератор признаков, сравнил его с ручными наборами на LightGBM и оценил устойчивость различий с помощью парного bootstrap по пользователям.

Результат

В контрольном запуске автоматический набор с сохранёнными параметрами Optuna получил NDCG@10 **0,09777**, ручной - **0,09507**. Точечная разность составляет **+2,84%** относительно ручной модели. 95% интервал разности включает ноль, поэтому превосходство статистически не подтверждено.



Статус: исследовательский эксперимент на сохранённых таблицах прежнего временного протокола. Исправление границ часовой метки реализовано в пересборке; полный прогон на новых таблицах ещё не выполнен.

github.com/w4ngle/recsys-auto-features · Документация: 1 ноября 2026

01 / Задача и реализация

Постановка

Исходные данные - музыкальные прослушивания и лайки Yambda-50M. Я выбираю 5000 пользователей со всей историей, seed=42. Строка представляет прослушивание; label=1, если первый лайк пары пользователь/трек произошёл в течение следующего часа.

Задача - упорядочить наблюдённые события по вероятности лайка. Это не полный поиск рекомендаций в каталоге: кандидаты определены реальными прослушиваниями, а повторные пары не дедуплицируются. Ранее лайкнутые до среза пары исключаются.

Генерация признаков

Подход	Сущности и окна	Признаков
Ручной основной	Пользователь, трек, пара; 1/7/30 дней и all-time	37
Ручной расширенный	Дополнительно пользователь-артист и пользователь-альбом	61
Автоматический	Пять сущностей; 1/3/7/14/21/30/60/90/180 дней и all-time	151

Шаблон генерации: сущность × событие × окно. Для комбинаций рассчитываются счётчики, конверсии likes/(listens+1), средняя доля прослушивания для пользователя и трека. Неиспользуемые константные ui_likes и ui_cr исключаются из автоматического набора.

Модели и сравнение

Основной эксперимент использует LGBMClassifier: вероятности бинарного события служат оценками ранжирования. Для каждого набора я сравниваю стандартные параметры и сохранённые параметры исторического Optuna-поиска. Выбор вариантов выполняется по val.

В ранних итерациях также исследовались LambdaRank, top-37 по gain и отсев коррелирующих признаков. Эти эксперименты сохранены отдельно и не переобучались в контрольной серии.

Реализованные компоненты

Подготовка меток и срезов, два генератора признаков, обучение LightGBM, пользовательские метрики, парный bootstrap, новый протокол Optuna, последовательный ноутбук, проверки на синтетических данных и сохранение метаданных эксперимента.

02 / Эксперименты и выводы

Контрольный запуск выполнен 31 октября 2026 на прежних таблицах. Train: 2 349 940 строк, val: 1 309 444, test: 1 333 420. На test 4941 пользователь после фильтров; NDCG/Recall усредняются по 1656 пользователям с положительной меткой.

Модель	NDCG val	NDCG test	Recall test
Ручная, стандартная	0,083245	0,097539	0,124424
Ручная, Optuna params	0,094114	0,095070	0,125752
Авто, стандартная	0,085212	0,091007	0,119185
Авто, Optuna params	0,089725	0,097770	0,123910

Статистическая проверка

Среди вариантов каждой семьи по val выбираются сохранённые параметры Optuna. Для их сравнения я рассчитываю NDCG один раз по каждому пользователю и пересэмплирую парные разности. Используются 2000 повторов, seed=42, percentile-интервал 95%. Точные ничьи разрешаются порядком входных строк.

Разность NDCG test: **+0,002700**; среднее bootstrap: **+0,002617**; 95% CI: **[-0,004240; +0,009805]**. Автоматический вариант выше по NDCG и ниже по Recall. Превосходство, эквивалентность и non-inferiority не установлены.

Optuna и происхождение результатов

В контрольной серии я повторно применяю сохранённые best_params. Исторический JSON не содержит полного журнала trials, seed и бюджета, поэтому исходный поиск нельзя восстановить целиком. Ранние NDCG test 0,0883/0,0909 и текущие 0,0951/0,0978 относятся к разным сериям и не объединяются.

Для следующего поиска реализован общий протокол: по умолчанию 30 trials на набор, одинаковое пространство, TPE seed=42, последовательное выполнение, цель NDCG@10 val. Early stopping - logloss val. Test в подбор не передаётся; история сохраняется в SQLite и CSV. Полный новый поиск ещё не выполнен.

Что показал проект

Шаблонная генерация позволяет систематически исследовать временные окна. Увеличение числа признаков не гарантирует улучшения; важность gain не доказывает причинный вклад. Текущие результаты обосновывают дальнейшее исследование, но не утверждение о доказанной победе автоматики.

03 / Качество и воспроизведение

Временная корректность

Признаки используют историю строго до среза. При проверке протокола я выявил отдельную проблему: метка требует часа после события, а прежние таблицы не оставляли зазор у границ частей и конца наблюдений. Это может переносить информацию о метках через границу и создавать незавершённые метки в конце данных.

В build_train/build_train_auto и prepare_data.py реализовано исключение последнего часа окон. Новые таблицы сохраняются в data/rebuilt/. Полноту лайков до конца прослушиваний нужно подтвердить на исходных данных. Полный прогон этого режима не выполнен; опубликованные метрики остаются результатами предыдущего протокола.

Проверки

Четыре тестовых метода проверяют метрики на известных ранжированиях и ничьих, некорректные score, постоянную разность bootstrap, совпадение общих признаков, cutoff и часовой зазор. Дополнительно проверен один Optuna trial на синтетических данных.

В трёх частях подтверждено совпадение uid/item_id/label и порядка строк ручных/автоматических таблиц. Сохранены параметры, версии пакетов, SHA-256 входных parquet и gain-важности. Небольшие выводы выполненного ноутбука доступны без загрузки датасета.

Как воспроизвести

1. Клонировать репозиторий, создать виртуальное окружение и установить requirements-checked.txt.
2. Выполнить python -m unittest discover -s tests.
3. Для прежних таблиц указать RECSYS_DATA_DIR и выполнить notebooks/T-Lab.ipynb.
4. Для нового временного протокола сначала запустить prepare_data.py и выбрать data/rebuilt/.
5. Для нового поиска включить RUN_SEARCH=True и уникальный STUDY_TAG.

Контрольное окружение: Python 3.14.0, LightGBM 4.7.0. Полные версии основных пакетов и команды доступны в docs/reproduction.md. Данные и локальные модели не включены в Git; точные требования к памяти и времени не измерены.

Следующие шаги

Пересборка с зазорами и проверка полноты данных; одинаковый новый Optuna-поиск; несколько seed пользователей; новый временной test. Для каталожных рекомендаций потребуется генерация кандидатов и оценка уникальных треков. Онлайн-эффект и промышленное масштабирование не исследовались.

Источники: Yambda (huggingface.co/datasets/yandex/yambda); CSV/JSON и код в репозитории. Подробности: docs/methodology.md, docs/results.md, AUDIT.md.